

MOTION OS

Compose Motion & Pixels

Choosing the right API · live demos in the app



Abdul Wahab

September 2026

This deck is one `updateTransition`. Every flourish names its API.

GOAL

Leave knowing which API

- 1 Which API for which problem — not a catalog dump
- 2 When Canvas / custom Layout is actually worth it
- 3 Shared performance rules we can ship with

Five beats, then Q&A

- 1 Animation decision tree + live proof
- 2 Graphics & Layout tree + live proof
- 3 Performance checklist
- 4 Apply to our apps
- 5 Q&A

When not to animate

- Prefer system / Material defaults first
- Motion must serve hierarchy or feedback
- If it fights readability or a11y — cut it
- Games and Canvas are rare in product UI

Custom is for moments that sell the product.

Need motion?

Tap a branch we will prove

- └─ Appear / disappear? → `AnimatedVisibility`
- └─ Swap content / screens? → `AnimatedContent`
- └─ Many props, one state? → `updateTransition`
- └─ Imperative / interruptible? → `Animatable`
- └─ Gesture / organic settle? → `spring()`
- └─ Authored “story” timing? → `keyframes`
- └─ List ↔ detail continuity? → `SharedTransition`
- └─ Per-frame draw / game? → `withFrameNanos + Canvas`

Live app: tappable branches jump to proof. PDF: pattern only.

Intent, not decoration

tween

Directed, timed · enter / morph

Use when: structure transitions, screen changes, sequential reveals.

Key: `durationMillis`, `easing`

spring

Physical · drag release, playful

Use when: gesture settle, interactive drag, playful taps.

Key: `stiffness`, `dampingRatio`

keyframes

Hand-authored · overshoot, pauses

Use when: cinematic story, race-then-settle, overshoot.

Key: `at / using / durationMillis`

infinite

Loop · ambient, loaders

Use when: spinners, pulsing indicators (sparingly).

⚠ Not for hero UI — it fights user attention.

Animation APIs — what they do

APPEAR & SWAP

AnimatedVisibility	Show or hide with Enter / Exit
AnimatedContent	Swap screens or content (A → B)
Crossfade	Fade-only content swap
animateContentSize	Animate size when children change

DRIVE VALUES

animate*AsState	One property toward one target
updateTransition	Many properties, one discrete state
Animateable	Imperative: snap, stop, sequence, cancel
rememberInfiniteTransition	Looping values — loaders, ambient
animateDecay	Fling from remaining velocity

SPECS · INTENT

tween	Directed, timed — enter, morph, structure
spring	Physics settle — drag release, playful tap
keyframes	Authored story — race, pause, overshoot
snap	Jump with no interpolation
repeatable	Play N times, or forever

CONTINUITY

SharedTransitionLayout	Scope wrapping both destinations
sharedElement	The same visual object travels
sharedBounds	Bounds morph; content inside can differ

PER-FRAME

withFrameNanos	Vsync clock — simulation, games, orbits
-----------------------	---

Not every function in `androidx.compose.animation` — the set we choose from. Tree picks; table names.

AnimatedVisibility

- One boolean. Enter plays in, Exit plays out.
- Combine transitions with + — `fadeIn()` + `expandVertically()`.
- Error banners, optional fields, tooltips — this is most of our motion.

[AnimatedVisibility](#)[EnterTransition](#)[ExitTransition](#)

```
AnimatedVisibility(visible = state) {  
    Card { /* content */ }  
}
```

THREE ENTER / EXIT PAIRS

fadeIn() + fadeOut()

opacity only · smooth swap

slideIn + slideOut

directional · toolbars, banners

expandVertically + shrinkVertically

optional fields, error messages

Combine with `+: fadeIn() + expandVertically()` gives both at once

`showcase/AnimatedVisibilityDemo.kt`

PROOF

Morphing Action Button

- One state. Width, color, glyph stay in lockstep.
- Tween the structure. Spring the glyph.
- Pay / save / sync in our apps.

updateTransition

animateDp

PathMeasure

graphicsLayer

```
updateTransition(state)
animateDp { Idle → 280.dp; Loading → 66.dp }
textAlpha delayMillis = 120 // structure leads
```

STATE MACHINE

Idle

w = 280 dp
label visible

tap →

Loading

w = 66 dp
spinner · no label

done →

Done

w = 66 dp
check ✓

structure → tween · label → tween + 120 ms delay · glyph → spring

showcase/components/MorphingActionButton.kt

PSEUDOCODE

updateTransition — morphing button

one state → many properties

updateTransition(buttonState)

```
width    ← tween      Idle / Loading / Done
color    ← tween
label    ← tween + 120ms delay  // structure leads
glyph    ← spring      // micro-interaction
```

Full source in Android Studio

app/.../showcase/components/MorphingActionButton.kt

PROOF

Keyframes Theme Wipe

- Tap to wipe light ↔ dark.

Animatable

keyframes

clipRect

```
keyframes {  
    0.90f at 520 using cinematic  
    overshoot at 700  
    target at 850  
}
```

ANIMATION TIMELINE · 850 ms total

0 → 0.90
0 – 520 ms

Race (cinematic easing)

→ 1.03
700 ms

Overshoot

→ 1.00
850 ms

Settle

clipRect(width × fraction) — draw primitive, not layout

showcase/KeyframesThemeWipeDemo.kt

PSEUDOCODE

keyframes + clipRect wipe

Animatable fraction 0 → 1

keyframes:

 race to 90% @ 520ms

 overshoot @ 700ms

 settle @ 850ms

clipRect(width × fraction) // draw, not layout

[Full source in Android Studio](#)

app/.../showcase/KeyframesThemeWipeDemo.kt

PROOF

Shared Element Gallery

- The card travels — it doesn't swap.
- Keys: swatch-id, title-id.
- Compose's answer to hero transitions.

SharedTransitionLayout

sharedElement

AnimatedContent

```
SharedTransitionLayout {  
    Modifier.sharedElement(  
        rememberSharedContentState("swatch-$id")  
    )  
}
```

HOW THE CARD TRAVELS

List

```
sharedElement("swatch-$id")  
sharedElement("title-$id")  
  
compact, in row
```

travels —→

Detail

```
same keys  
same scope  
  
expands in place
```

stable unique keys · AnimatedContent as scope · don't share huge subtrees

showcase/SharedElementGalleryDemo.kt

Draw or place?

Custom drawing or layout?

- └ Tint / shape / clip only? → Modifier / Shape
- └ Charts, gauges, game art? → Canvas + DrawScope
- └ Children in custom positions? → custom Layout
- └ Deferred / dependent measure? → SubcomposeLayout

Canvas is not a layout system. Layout is not a drawing API.

Canvas Pulse Chart

- Designer gives you a custom chart? This is where you land.
- One fraction. Many properties.
- Same cinematic pattern as UI morphs.

Canvas

Path

Animatable

stagger

```
val t = remember { Animatable(0f) }
```

```
Canvas { drawPath(area, brush) }
```

```
localT = ((t - i * 0.06f) / 0.7f)
```

DRAWSCOPE BREAKDOWN

```
DrawScope {  
    drawLine        → grid lines  
    drawPath        → area fill + stroke  
    drawRoundRect    × n → bars (stagger)  
}
```

one Animatable(0→1) drives all · local stagger: $(t - i \times 0.06) / 0.7$

showcase/CanvasPulseChartDemo.kt

LAYOUT

Fan Deck

- Layout places. Canvas draws the ring.
- `zIndex = frontness`. `graphicsLayer` sells depth.
- Swipe / tap — spring settle. Product: wallet, stories, picker.

Layout

`placeRelative`

`zIndex`

`graphicsLayer`

`spring`

```
val a = PI/2 + 2π * (i - selected) / n
p.placeRelative(x, y, zIndex = frontness)
// graphicsLayer { rotationY = ...; scale = depth }
```

LAYER SPLIT

Layout { } ⇨ position only
`placeRelative(x, y)`
`zIndex = frontness`

graphicsLayer { } ⇨ visual depth
`rotationY, scale`

Canvas { } ⇨ draws the ring

spring settle on swipe -- same `spring()` as any UI interaction

`showcase/OrbitalMenuLayoutDemo.kt`

PSEUDOCODE

custom Layout — fan deck

Layout:

measure each child

place on ellipse(selected)

zIndex = frontness // who is in front

graphicsLayer: rotationY, scale // depth, not placement

Canvas: draw the ring

spring settle after swipe

[Full source in Android Studio](#)

app/.../showcase/OrbitalMenuLayoutDemo.kt

Same toolbox. Different intensity.

A · CINEMATIC UI

Aurora Unlock

Hold the core. Animatable, springs, shared clock.

B · GAME LOOP

Neon Rush

withFrameNanos racer. Same Canvas primitives. Not default product UI.

Same behavior. Fewer moving parts.

BEHAVIOR	XML / VIEW	PARTS	COMPOSE	PARTS
Show / hide	<code>View.GONE</code> ↔ <code>VISIBLE</code> + <code>TransitionManager</code>	3	<code>AnimatedVisibility(visible) { }</code>	1
Async button morph	<code>AnimatorSet</code> + 4× <code>ObjectAnimator</code> + <code>Listener</code>	7	<code>updateTransition</code> – every property chained	1
Custom chart	<code>Custom View</code> + <code>onDraw()</code> + <code>ValueAnimator</code>	4	<code>Canvas { }</code> + one <code>Animatable</code>	2
List → detail continuity	<code>MotionScene</code> + 2+ <code>ConstraintSets</code> + <code>Transition</code> tag	4	<code>SharedTransitionLayout</code> + matching keys	2
Radial / fan picker	<code>Custom ViewGroup</code> : <code>constructors</code> + <code>onMeasure</code> + <code>onLayout</code>	5	One <code>Layout { measurables → placeRelative }</code> block	1
Screen transitions	4 <code>anim</code> XML + 4 <code>attrs</code> on every graph <code><action></code>	4	<code>transitionSpec</code> on <code>NavDisplay</code> – once for the graph	2

Moving parts = distinct objects to create, wire, and keep in sync. Not line counts.

Performance checklist

- 1 `graphicsLayer / draw / offset { }` — not layout every frame
- 2 Cache Path / paints in hot 60fps loops
- 3 One shared clock for coordinated effects
- 4 Don't animate a state read in every parent
- 5 Shared elements: stable keys, small regions
- 6 Shaders / AGSL: API guards + fallbacks

Motion moments → right API

MOMENT	API
Primary CTA async	<code>Morphing button / updateTransition</code>
List → detail	<code>Shared elements</code>
Theme / onboarding flourish	<code>keyframes</code> (sparingly)
Analytics widget	<code>Canvas chart</code>
Radial / fan picker	<code>custom Layout</code> (if design requires)
Everything else	<code>Material + Visibility / Content</code>

Canvas / Layout only when Material + standard animation can't express the design.

The lab is this repo

- 1 Compose Lab → `app/.../showcase/`
- 2 Handout → `docs/compose-lab-handout.md`
- 3 PDF deck → `docs/talk/compose-motion-pixels.pdf`
- 4 github.com/abdul-wahab-9d/MotionOS

No tablet? Phone layout is a tap-through lab.

QUICK REFERENCE

One page. Every API. No excuses.

ANIMATION — WHICH?

Appear / disappear?	AnimatedVisibility
Swap content / screens?	AnimatedContent
Multi-prop, one state?	updateTransition
Imperative / cancel?	Animatable
Gesture settle?	spring()
Cinematic story?	keyframes
Screen continuity?	SharedTransition
Game / per-frame?	withFrameNanos

GRAPHICS & LAYOUT — WHICH?

Tint / clip / shape?	Modifier / Shape
Charts / pixels / game?	Canvas + DrawScope
Custom child positions?	Layout
Dependent measure?	SubcomposeLayout

SPEC — WHAT INTENT?

tween	directed , timed
spring	physics settle
keyframes	hand-authored story
snap	no interpolation
infinite	ambient loop (sparingly)

WHEN JUST MATERIAL?

Standard transition	AnimatedVisibility
Size change	animateContentSize
Single value	animate* AsState
Fade swap	Crossfade

PERFORMANCE ✓ DO THIS

- ✓ [graphicsLayer](#) / [draw](#) / [offset](#) — not recompose per frame
- ✓ Cache Path & Paint in hot draw loops
- ✓ One shared clock for coordinated effects
- ✓ Stable unique keys + small regions (shared elements)
- ✓ API guard + fallback before [AGSL](#) / [shaders](#)

PERFORMANCE ✗ AVOID

- ✗ Animate state read in every parent
- ✗ Canvas to fake layout / child positioning
- ✗ Layout in a hot 60fps draw loop
- ✗ Huge subtrees in shared elements
- ✗ infinite transitions in hero UI

Your motion bugs, please

Where have we fought motion jank?

Any design ask that forced Canvas prematurely?

Who wants a follow-up on Nav + shared elements?

LAYOUT

Measure children, then `placeRelative` — that's custom layout from scratch. Canvas is for pixels, not for placing children.

ANIMATION

Visibility you know. Custom means `Animatable` + a spec. Story timing is keyframes. Physics is spring. Continuity across screens is shared elements.

THANK YOU

TEAM LEAD

Hammad Nawaz

and the team

Animatable

spring

keyframes

sharedElement

Canvas

Layout

Motion OS — tablet = deck

- 1 Clone the repo and run the app in Android Studio
- 2 Tablet or Pixel Tablet AVD (shortest side ≥ 600 dp) opens this talk
- 3 Phone opens Compose Lab — the demo list
- 4 Clicker: volume / D-pad / space · notes: N

Demos in this PDF are stills. The motion lives in the app.